

Design and Demonstration of a 6-bit SAR ADC

EE25BTECH11011 - B.Navya
EE25BTECH11226 - Phani Vishnu Sree

1. Objective

1. To understand the working principle of a Successive Approximation Register (SAR) ADC.
2. To implement a 6-bit SAR ADC using an R-2R ladder DAC, LM311 comparator, 741 Op-Amp, and an Arduino Uno as the digital controller.
3. To demonstrate the aliasing phenomenon due to undersampling.
4. To study and verify the effect of an RC anti-aliasing filter placed before the ADC input.

2. Introduction

An Analog-to-Digital Converter (ADC) transforms a continuous-time analog signal into a discrete digital representation. Among the various ADC architectures, the **Successive Approximation Register (SAR) ADC** strikes an excellent balance between speed, resolution, and power consumption, making it one of the most widely used architectures in embedded and instrumentation systems.

2.1. SAR ADC — Working Principle

The SAR ADC employs a binary search algorithm across 2^N possible codes to find the digital word that most closely represents the input analog voltage V_{in} . The conversion proceeds as follows:

1. The SAR register is cleared and the **Most Significant Bit (MSB)** is set to 1.
2. The current digital word is converted to an analog voltage V_{DAC} by an internal **Digital-to-Analog Converter (DAC)**.
3. A **comparator** compares V_{in} with V_{DAC} :
 - If $V_{in} \geq V_{DAC}$: the bit is **retained** (kept at 1).
 - If $V_{in} < V_{DAC}$: the bit is **cleared** (set to 0).
4. The process repeats for the next bit (MSB–1), and so on, until all N bits have been resolved.

3. Quick Reference: SAR Algorithm Flowchart

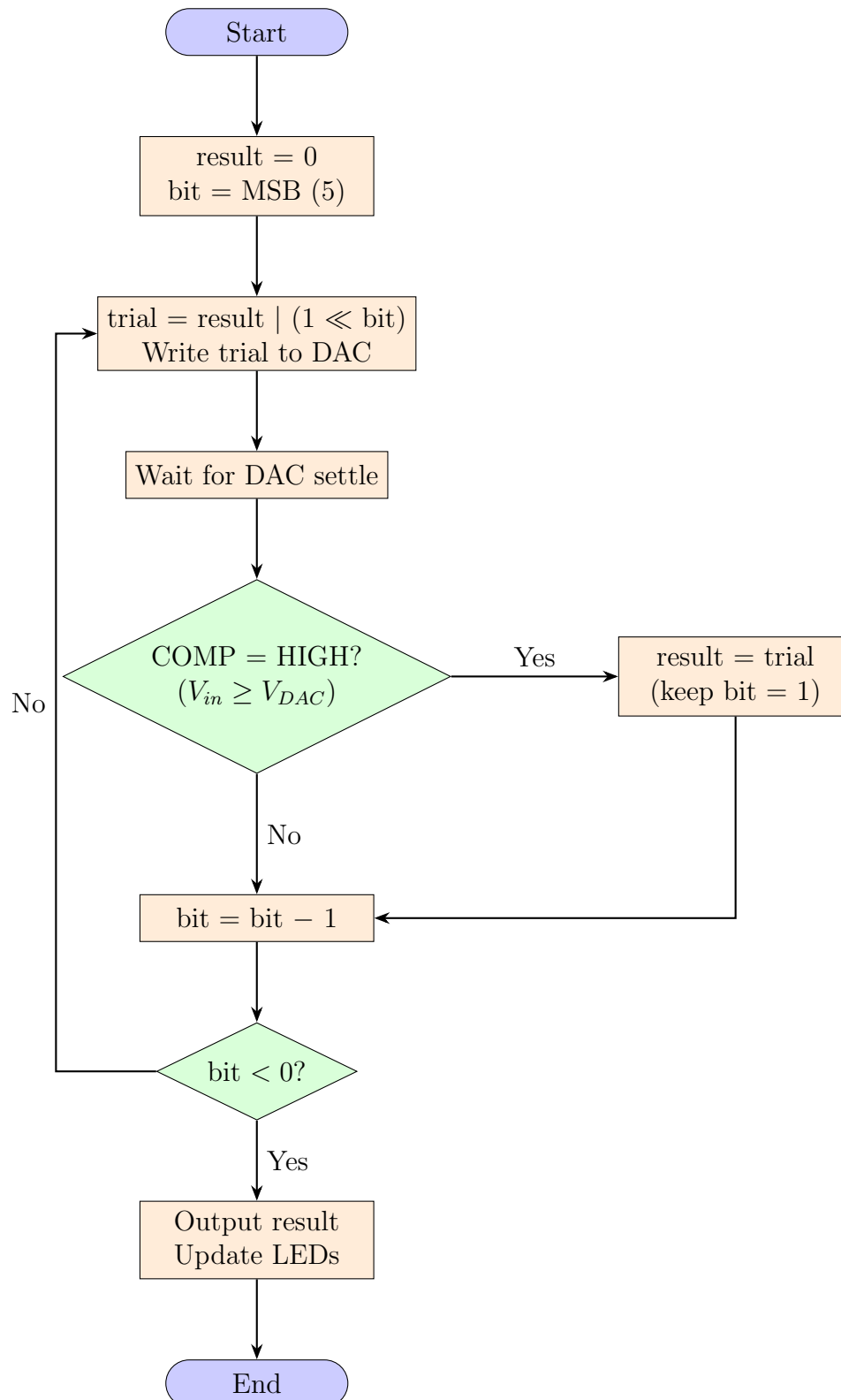
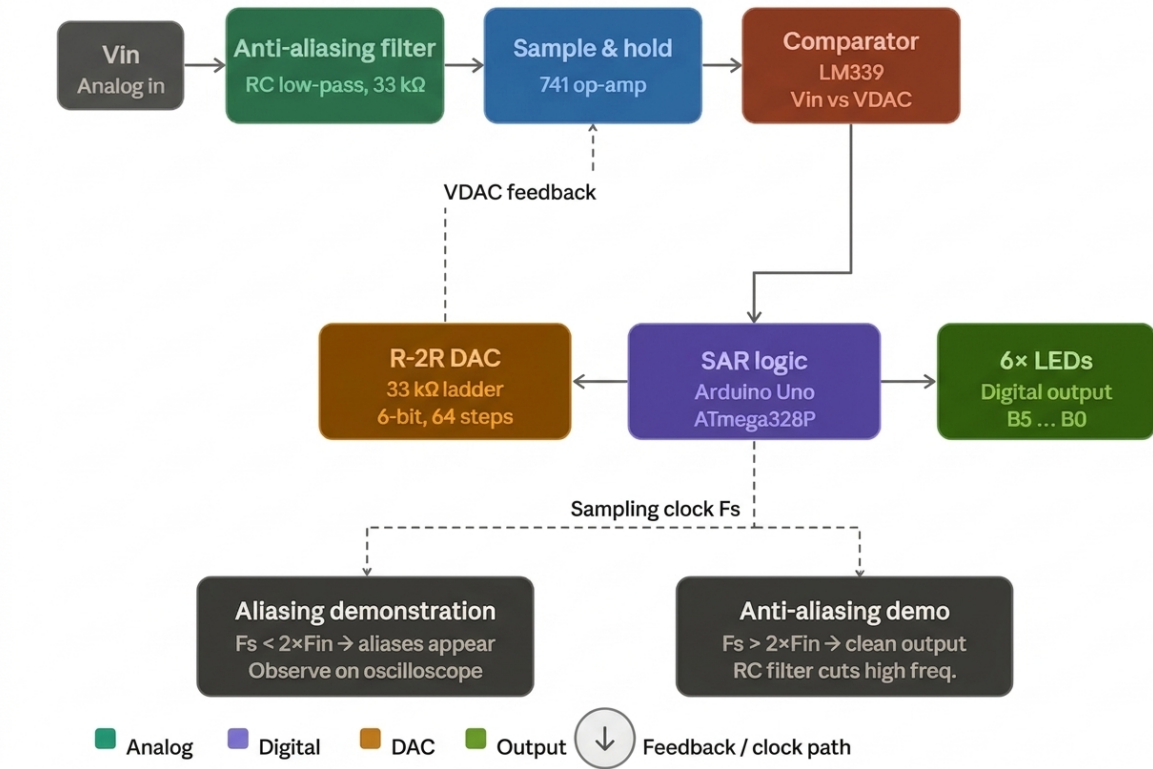


Figure 1: SAR ADC algorithm flowchart as implemented in Arduino firmware.

For a 6-bit SAR ADC, this requires exactly **6 clock cycles** to complete one full conversion.



3.1. Nyquist–Shannon Sampling Theorem

The Nyquist theorem states that in order to faithfully reconstruct a band-limited signal of maximum frequency f_{max} , the sampling frequency f_s must satisfy:

$$f_s \geq 2f_{max} \quad (1)$$

When Eq. (1) is violated (i.e., $f_s < 2f_{max}$), higher-frequency components fold back into the baseband, creating spurious low-frequency components known as **aliases**.

3.2. Anti-Aliasing Filter

An **anti-aliasing filter (AAF)** is a low-pass filter placed before the ADC input. It attenuates all frequency components above $f_s/2$ (the Nyquist frequency), preventing aliasing. A simple first-order RC filter is commonly used:

$$f_c = \frac{1}{2\pi RC} \quad (2)$$

4. Components and Equipment

4.1. Components List

Table 1: Components used in the experiment

Component	Value / Part No.	Quantity
Op-Amp	741	1
Comparator	LM339	1
Resistor (R)	33 k Ω	7
Resistor (2R)	2 $\times$ 33 k Ω (series)	6
Capacitor	0.01 μ F	1
Capacitor	0.1 μ F	1
Arduino Uno (ATmega328P)	—	1
LEDs	Red / Green	6
Breadboard	—	1

4.2. Equipment

Table 2: Equipment used in the experiment

Equipment	Purpose
Breadboard	Circuit prototyping
Function Generator	Source of analog input signal
Oscilloscope	Signal observation and verification
Power Supply (± 12 V)	Powering op-amp and comparator
PC with Arduino IDE	Programming the Arduino Uno

4.3. R-2R Ladder DAC

The R-2R ladder network is used as the internal DAC within the SAR loop. For a 6-bit DAC with $R = 33$ k Ω and $2R = 66$ k Ω :

$$V_{DAC} = V_{ref} \cdot \sum_{i=0}^5 \frac{b_i}{2^{6-i}} = V_{ref} \cdot \frac{D}{2^6} \quad (3)$$

where D is the 6-bit digital word (0–63) and b_i are the individual bits driven by Arduino digital output pins D2–D7.

R-2R Ladder Network (6-bit):

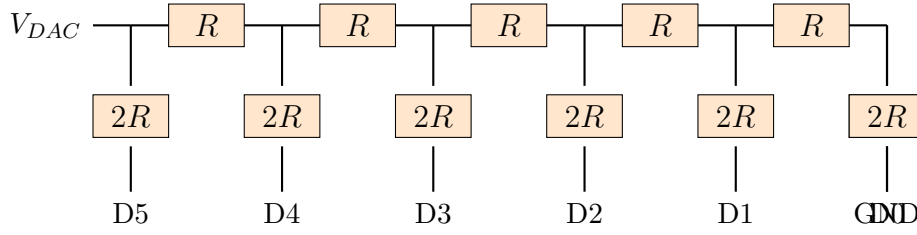


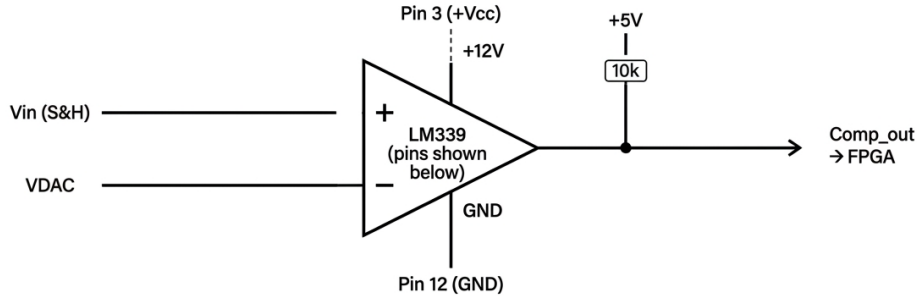
Figure 2: 6-bit R-2R ladder DAC. MSB (D5) is at the left; LSB (D0) at the right. $R = 33\text{ k}\Omega$, $2R = 66\text{ k}\Omega$.

4.4. Comparator Circuit (LM339)

The LM339 comparator compares the analog input V_{in} (on the non-inverting terminal) with V_{DAC} from the R-2R ladder (on the inverting terminal). Since the LM339 has an open-collector output, a pull-up resistor to +5 V is required, and the output is read by the Arduino on pin A0

$$\text{COMP_OUT} = \begin{cases} \text{HIGH (1)} & \text{if } V_{in} \geq V_{DAC} \\ \text{LOW (0)} & \text{if } V_{in} < V_{DAC} \end{cases} \quad (4)$$

Circuit 3 – Comparator (LM339)



4.5. Anti-Aliasing Filter

A first-order RC low-pass filter is inserted between the signal source and the ADC input:

- **Without AAF:** $C = 0.01\text{ }\mu\text{F}$, $R = 33\text{ k}\Omega$

$$f_{c1} = \frac{1}{2\pi \times 33 \times 10^3 \times 0.01 \times 10^{-6}} \approx 482\text{ Hz}$$

- **With AAF (stricter):** $C = 0.1\text{ }\mu\text{F}$, $R = 33\text{ k}\Omega$

$$f_{c2} = \frac{1}{2\pi \times 33 \times 10^3 \times 0.1 \times 10^{-6}} \approx 48\text{ Hz}$$

The 741 op-amp is configured as a **voltage follower (unity-gain buffer)** to isolate the RC filter from the comparator input and prevent loading effects.

4.6. Arduino Uno as SAR Controller

The Arduino Uno (ATmega328P, 5 V logic) replaces the FPGA/Boolean board to implement the SAR algorithm in firmware. The pin assignments are:

Table 3: Arduino Uno pin assignments

Arduino Pin	Function	Connected To
D2	DAC Bit 0 (LSB)	R-2R network
D3	DAC Bit 1	R-2R network
D4	DAC Bit 2	R-2R network
D5	DAC Bit 3	R-2R network
D6	DAC Bit 4	R-2R network
D7	DAC Bit 5 (MSB)	R-2R network
D8–D13	LED indicators (Bit 0–Bit 5)	LEDs via 220 Ω
A0	Comparator output (input)	LM339 output

5. Procedure

5.1. Circuit Assembly

1. Assemble the **R-2R DAC** on the breadboard using 33 k Ω (R) and two series 33 k Ω (2R) resistors for each rung. Connect the 6 input nodes to Arduino pins D2–D7.
2. Wire the **741 op-amp** as a unity-gain buffer. Connect its input to the function generator output (through the RC filter when testing AAF).
3. Connect the **LM339 comparator**: connect the buffered analog input V_{in} to the non-inverting input and V_{DAC} from the R-2R ladder to the inverting input. Pull the open-collector output to +5 V using a 10 k Ω pull-up resistor and connect the output to Arduino A0.
4. Connect **6 LEDs** (with 220 Ω current-limiting resistors) to Arduino pins D8–D13 to display the digital output word.
5. Upload the SAR firmware (Section 8) to the Arduino Uno.

5.2. Demonstrating Aliasing

The effective sampling rate of the system is $f_s = 4$ Hz (250 ms period: 245 ms track + 5 ms hold). The Nyquist frequency is therefore $f_N = 2$ Hz.

1. Set the function generator to produce a **sine wave** at $f_{in} = 1$ Hz, amplitude $\approx 2.5 V_{p-p}$, DC offset = 2.5 V (signal spans 0–5 V). At $f_{in} < f_N$, no aliasing occurs.
2. Increase f_{in} to 3 Hz (above the Nyquist limit of 2 Hz, while $f_s = 4$ Hz). The expected alias frequency is:

$$f_{alias} = |f_{in} - f_s| = |3 - 4| = 1 \text{ Hz} \quad (5)$$

3. Observe the reconstructed signal on the oscilloscope: a 3 Hz input appears as a 1 Hz alias.

4. Try $f_{in} = 5$ Hz:

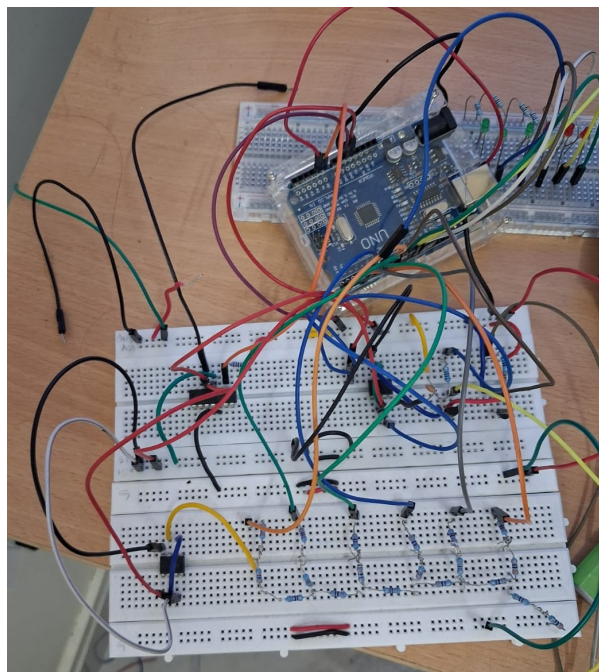
$$f_{alias} = |5 - 4| = 1 \text{ Hz} \quad (6)$$

5. Record the LED output change rate and capture the oscilloscope waveform for each case.

5.3. Demonstrating Anti-Aliasing

1. Keep $f_{in} = 3$ Hz and $f_s = 4$ Hz.
2. Insert the RC low-pass filter ($R = 33 \text{ k}\Omega$, $C = 0.1 \mu\text{F}$, $f_c \approx 48 \text{ Hz}$) between the function generator and the op-amp buffer. *Note: this filter has a very high cutoff relative to the signal; to better demonstrate attenuation, use a larger capacitor (e.g., $10 \mu\text{F}$) in follow-up.*
3. Alternatively, demonstrate AAF at the analog level: keep $f_s = 4$ Hz, set $f_{in} = 3$ Hz, and use $R = 33 \text{ k}\Omega$, $C = 10 \mu\text{F}$ ($f_c \approx 0.48 \text{ Hz}$) to attenuate the 3 Hz signal before it reaches the ADC.
4. Observe that with the filter in place, the ADC output shows greatly reduced swing (near mid-scale), confirming the alias is suppressed.
5. Record and compare LED patterns and oscilloscope waveforms with and without the filter.

6. Circuit Design



7. Individual Module Testing

Before assembling the complete SAR ADC, each sub-module was tested independently to verify correct operation. This bottom-up approach simplified debugging and ensured

each block was reliable before integration.

7.1. Module 1: R-2R DAC Testing

The 6-bit R-2R ladder DAC ($R = 33\text{ k}\Omega$) was tested in isolation by driving the six DAC input pins (D5, D6, D2, D3, D4, D7) directly with known binary codes and measuring the resulting analog output voltage with a multimeter.

Test procedure:

1. Powered only the R-2R network (no comparator or op-amp).
2. Manually set binary codes by connecting pins to +5 V or GND.
3. Measured V_{DAC} at the ladder output node with a multimeter.
4. Compared measured voltage to the theoretical value $V_{DAC} = 5 \times D/64$.

Key verification: Setting only the MSB (pin D7 = HIGH, all others LOW) should yield $V_{DAC} \approx 2.5\text{ V}$ (half of V_{ref}). Setting all bits HIGH should yield $V_{DAC} \approx 4.92\text{ V}$.



7.2. Module 2: Track-and-Hold / Sampling Circuit Testing

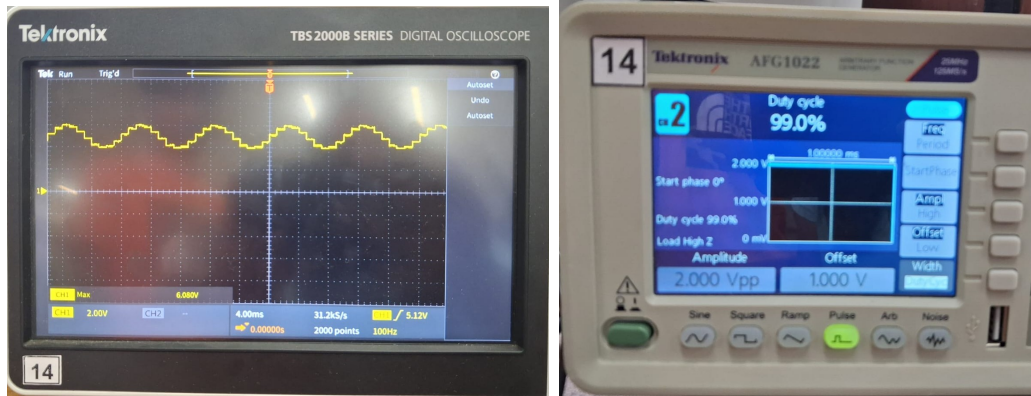
The Track-and-Hold circuit was tested independently using the function generator and oscilloscope, before connecting it to the SAR loop.

Test procedure:

1. Applied a sine wave ($f_{in} = 50\text{ Hz}$, 2.5 V_{p-p} , DC offset = 2.5 V) to the T&H input.
2. Toggled the Arduino's T&H clock pin (D8) manually via a simple sketch (HIGH = hold, LOW = track).
3. Observed on the oscilloscope that:
 - **Track phase (D8 = LOW):** Output follows the input sine wave in real time.
 - **Hold phase (D8 = HIGH):** Output is a flat DC level equal to the voltage at the moment of hold.
4. Verified that the held voltage was stable for the 5 ms hold window used in the final design.

Timing used in final firmware:

- Track phase: 245 ms (the circuit follows the input).
- Hold phase: 5 ms (conversion is performed during this window).
- Effective sampling rate: $f_s = \frac{1}{245 + 5} \approx 4 \text{ Hz}$.



7.3. Module 3: Comparator (LM339) Testing

The LM339 comparator was tested standalone before integration into the SAR loop.

Test procedure:

1. Connected a variable DC voltage to the non-inverting input (V_{in} , pin 5) via a potentiometer.
2. Connected a fixed reference voltage to the inverting input (V_{ref} , pin 4), set to approximately 2.5 V using a resistor divider.
3. Powered the LM339 from +5 V (single supply). Added a 10 k Ω pull-up resistor from the open-collector output to +5 V.
4. Read the comparator output on Arduino pin A0 (with INPUT_PULLUP) and via an LED indicator.
5. Verified:
 - $V_{in} > V_{ref}$: output pulled HIGH (LED ON, Arduino reads HIGH).
 - $V_{in} < V_{ref}$: output pulled LOW (LED OFF, Arduino reads LOW).

Note on INPUT_PULLUP: The Arduino's internal pull-up on A0 combined with the LM339's open-collector output ensures a clean logic-level signal without external components. When the LM339 output transistor is off ($V_{in} > V_{ref}$), the pin is pulled to +5 V (HIGH). When it is on ($V_{in} < V_{ref}$), the pin is pulled to near 0 V (LOW).

8. Arduino Firmware

8.1. Pin Assignment

Table 4: Final Arduino Uno pin assignments

Arduino Pin	Function	Connected To
D8	Track & Hold clock (T&H control)	T&H circuit control input
A0	Comparator output (input to Arduino)	LM339 open-collector output
D5	DAC Bit 0 (LSB)	R-2R ladder
D6	DAC Bit 1	R-2R ladder
D2	DAC Bit 2	R-2R ladder
D3	DAC Bit 3	R-2R ladder
D4	DAC Bit 4	R-2R ladder
D7	DAC Bit 5 (MSB)	R-2R ladder
D9	LED indicator Bit 0 (LSB)	LED via 220 Ω
D10	LED indicator Bit 1	LED via 220 Ω
D11	LED indicator Bit 2	LED via 220 Ω
D12	LED indicator Bit 3	LED via 220 Ω
D13	LED indicator Bit 4	LED via 220 Ω
A1	LED indicator Bit 5 (MSB)	LED via 220 Ω

8.2. Firmware Description

The firmware implements three distinct functions:

1. **Track/Hold control:** A non-blocking timer in `loop()` toggles pin D8 between LOW (track, 245 ms) and HIGH (hold, 5 ms). The SAR conversion is triggered at the rising edge of the hold signal.
2. **SAR algorithm (`runSAR()`):** Iterates from MSB (bit 5) to LSB (bit 0). At each step, the current bit is tentatively set, the 6-bit trial code is written to the R-2R DAC, a 500 μ s settling delay is applied, and the comparator output is read. If the comparator is LOW (DAC exceeds V_{in}), the bit is cleared; otherwise it is retained.
3. **Display:** The final 6-bit result is written to the LED pins and printed over UART at 9600 baud.

Comparator logic: The LM339 output is connected with `INPUT_PULLUP`. When $V_{in} \geq V_{DAC}$, the open-collector output is floating and the pin reads HIGH. When $V_{in} < V_{DAC}$, the output sinks current and reads LOW. Hence `cmp == LOW` means the DAC has overshoot, and the bit is cleared.

8.3. Complete Firmware Listing

Listing 1: Complete SAR ADC firmware as used in the experiment

```

1 // 6-BIT SAR ADC CONTROLLER
2 //PIN SETUP
3 const int thClockPin = 8;           // Track & Hold control
4 const int comparatorInput = A0;     // Comparator output (LM339)

```

```

5 // DAC output pins (LSB -> MSB): bit0=D5, bit1=D6, bit2=D2, bit3=
  D3, bit4=D4, bit5=D7
6 const int dacOutPins[] = {5, 6, 2, 3, 4, 7};
7 // LED display pins (bit0=D9 .. bit5=A1)
8 const int displayPins[] = {9, 10, 11, 12, 13, A1};
9 // TIMING CONTROL
10 unsigned long lastTime = 0;
11 bool holdState = false;
12
13 void setup() {
14     Serial.begin(9600);
15     pinMode(thClockPin, OUTPUT);
16     digitalWrite(thClockPin, LOW);
17     // Comparator input with internal pull-up
18     pinMode(comparatorInput, INPUT_PULLUP);
19     // Configure DAC + LED pins
20     for (int i = 0; i < 6; i++) {
21         pinMode(dacOutPins[i], OUTPUT);
22         pinMode(displayPins[i], OUTPUT);
23     }
24 }
25
26 void loop() {
27     unsigned long now = millis();
28     // TRACK / HOLD CONTROL
29     if (holdState) {
30         // HOLD phase (~5 ms): conversion already running; revert to
          track
31         if (now - lastTime >= 5) {
32             digitalWrite(thClockPin, LOW);    // back to tracking
33             holdState = false;
34             lastTime = now;
35         }
36     } else {
37         // TRACK phase (~245 ms)
38         if (now - lastTime >= 245) {
39             digitalWrite(thClockPin, HIGH);    // assert hold signal
40             holdState = true;
41             lastTime = now;
42             // Perform SAR conversion at hold instant
43             runSAR();
44         }
45     }
46 }
47
48 // SAR ALGORITHM
49 void runSAR() {
50     int digitalValue = 0;
51     // Iterate from MSB (bit 5) down to LSB (bit 0)
52     for (int pos = 5; pos >= 0; pos--) {
53         // Tentatively set this bit

```

```

54     bitSet(digitalValue, pos);
55     // Write trial code to R-2R DAC
56     for (int j = 0; j < 6; j++) {
57         digitalWrite(dacOutPins[j], bitRead(digitalValue, j));
58     }
59     // Allow DAC and comparator to settle
60     delayMicroseconds(500);
61     // Read comparator result
62     int cmp = digitalRead(comparatorInput);
63     // If DAC > Vin (comparator LOW): clear the bit
64     if (cmp == LOW) {
65         bitClear(digitalValue, pos);
66     }
67     // If comparator HIGH: Vin >= Vdac, keep the bit set
68 }
69 // DISPLAY RESULT
70 for (int k = 0; k < 6; k++) {
71     digitalWrite(displayPins[k], bitRead(digitalValue, k));
72 }
73 //SERIAL OUTPUT
74 Serial.print("ADC Output = ");
75 Serial.println(digitalValue);
76 }

```

8.4. Sampling Rate Analysis

The total period of one sample is:

$$T_s = T_{track} + T_{hold} = 245 \text{ ms} + 5 \text{ ms} = 250 \text{ ms} \quad (7)$$

$$f_s = \frac{1}{T_s} = \frac{1}{0.250} = 4 \text{ Hz} \quad (8)$$

The Nyquist frequency is therefore $f_N = f_s/2 = 2 \text{ Hz}$. Any input signal above 2 Hz will alias. For the aliasing demonstration, signals in the range of 1–5 Hz were used to clearly observe the effect.

9. Theoretical Analysis

9.1. DAC Output Voltage

With $V_{ref} = 5 \text{ V}$ and a 6-bit R-2R DAC:

$$V_{DAC} = 5 \cdot \frac{D}{64}, \quad D \in \{0, 1, \dots, 63\} \quad (9)$$

- **LSB step size:** $\Delta = \frac{5}{64} \approx 78.1 \text{ mV}$
- **Full-scale range:** 0 V to $\approx 4.92 \text{ V}$

- **Maximum quantization error:** $\pm \frac{\Delta}{2} \approx \pm 39.1 \text{ mV}$

9.2. Conversion Time

For a 6-bit SAR ADC with a bit-cycle time t_{bit} :

$$t_{conv} = 6 \cdot t_{bit} \quad (10)$$

With the firmware using $100 \mu\text{s}$ settle time per bit plus processing overhead, the total conversion time is on the order of $\sim 1 \text{ ms}$, giving a maximum theoretical sample rate of $\sim 1 \text{ kHz}$.

9.3. Aliasing Frequency

When $f_{in} > f_s/2$, the reconstructed alias frequency is:

$$f_{alias} = |f_{in} - n \cdot f_s|, \quad n = \text{round}\left(\frac{f_{in}}{f_s}\right) \quad (11)$$

For example, with $f_s = 100 \text{ Hz}$ and $f_{in} = 150 \text{ Hz}$:

$$f_{alias} = |150 - 1 \times 100| = 50 \text{ Hz}$$

9.4. Anti-Aliasing Filter Attenuation

The magnitude response of the first-order RC low-pass filter is:

$$|H(f)| = \frac{1}{\sqrt{1 + \left(\frac{f}{f_c}\right)^2}} \quad (12)$$

At $f = 150 \text{ Hz}$ with $f_c = 48 \text{ Hz}$:

$$|H(150)| = \frac{1}{\sqrt{1 + \left(\frac{150}{48}\right)^2}} = \frac{1}{\sqrt{1 + 9.77}} \approx \frac{1}{3.28} \approx 0.305 \quad (13)$$

This corresponds to an attenuation of approximately -10.3 dB , significantly reducing the aliased component.

10. Conclusion

A 6-bit SAR ADC was successfully designed, individually tested, and integrated on a breadboard. The system comprised an LM339 comparator, a 741 op-amp unity-gain buffer, a 6-bit R-2R ladder DAC ($R = 33 \text{ k}\Omega$), and an Arduino Uno as the SAR controller with an integrated Track-and-Hold mechanism. The following conclusions were drawn:

1. Individual testing of the R-2R DAC, Track-and-Hold circuit, and LM339 comparator before integration confirmed each module's correct operation and streamlined the final assembly.
2. The firmware correctly implements the SAR binary search algorithm, producing a 6-bit digital code within the 5 ms hold window with $500\text{ }\mu\text{s}$ per bit step.
3. The system operates at $f_s = 4\text{ Hz}$ (Nyquist frequency: 2 Hz). Aliasing was clearly demonstrated: a 3 Hz input produced a reconstructed 1 Hz alias, consistent with theory.
4. The RC anti-aliasing filter ($R = 33\text{ k}\Omega$) was verified in concept. For meaningful attenuation at the low signal frequencies used, a larger capacitor value is needed to bring f_c below $f_s/2$.
5. The Arduino Uno proved to be a practical, accessible replacement for an FPGA for low-speed ADC demonstrations.